STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: UI_001_ANGULAR
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: UI Development with Angular using AI	STATUS: In Progress

Practice Worksheet – UI Development with Angular using AI

Master modern responsive front-end engineering: JavaScript DOM validation, TypeScript interfaces, Angular built-in structural directives (*ngFor, [ngStyle]), and Reactive Forms with Async Pipes.

KEY LEARNING OBJECTIVES:

- Develop robust client-side form validation and DOM manipulation in pure JavaScript.
- Construct strongly typed TypeScript models and interfaces with access modifiers.
- Implement dynamic templates with Angular Directives (*ngFor, *ngIf, and [ngStyle]).
- Build reactive Angular forms with custom validation and RxJS Async Pipe consumption.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Review the front-end UI requirements and model the component state.
Step 2: Draft the TypeScript component class with dependency injection and lifecycle hooks.
Step 3: Write the HTML template integrating Angular directives and event bindings.
Step 4: Document form validation logic, observable streams, and CSS styling rules.

EXERCISE 5.1: 004 & 005: JAVASCRIPT CLIENT-SIDE FORM VALIDATION & DOM SANITIZATION

Problem Statement: Write a pure JavaScript validation script for a user login form with fields: `username` and `password`. Implement real-time validation: `username` must not be blank and must be at least 5 characters; `password` must contain at least 8 characters including one uppercase letter and one special symbol. Dynamically append error message spans to the DOM.

Reference Baseline / Code / Scenario:

```
function validateLoginForm() { const u = document.getElementById('username').value; ... }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] DOM Element Selectors & Events: _____

[2] Validation Regex & Length Rules: _____

[3] Dynamic Error Span Insertion: _____

EXERCISE 5.2: 006: TYPESCRIPT STRONGLY TYPED CUSTOMER MODEL & GREETER SERVICE

Problem Statement: Create a TypeScript interface `Customer` with attributes: `id: number`, `name: string`, `email: string`, `loyaltyTier: 'Silver' | 'Gold' | 'Platinum'`, and optional `discountCoupon?: string`. Create a service class `CustomerService` with a method `getWelcomeMessage(c: Customer): string` returning personalized markdown greetings.

Reference Baseline / Code / Scenario:

```
export interface Customer { id: number; name: string; email: string; loyaltyTier: 'Silver'|'Gold'|'Platinum'; }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] TypeScript Interface Specification: _____

[2] Service Class & Method Implementation: _____

[3] Type Assertion & Union Types: _____

EXERCISE 5.3: 007 & 008: ANGULAR DYNAMIC DIRECTIVES: *NGFOR AND [NGSTYLE]

Problem Statement: In an Angular component `QuestionListComponent`, display an array of exam questions. In the template: use `*ngFor` with `trackBy` to loop through questions. Use `[ngStyle]` to dynamically render the question difficulty badge: 'Easy' (green background), 'Medium' (amber background), and 'Hard' (rose background).

Reference Baseline / Code / Scenario:

STUDENT SOLUTION & CODE WORKSPACE:

[1] Component TypeScript Class & Array: _____

[2] HTML Template with *ngFor & trackBy: _____

[3] [ngStyle] Color Mapping Function: _____

EXERCISE 5.4: 009 & 010: ANGULAR REACTIVE FORMS & OBSERVABLE STREAMS WITH ASYNC PIPE

Problem Statement: Build an Angular `ProductFormComponent` using `FormBuilder`. Form fields: `productName` (required, minLength 3), `unitPrice` (required, min 1.0), and `sku` (custom async validator checking uniqueness). In the template, display live time stream using an RxJS `timer(0, 1000)` rendered via the `| async` pipe.

Reference Baseline / Code / Scenario:

```
this.productForm = this.fb.group({ productName: ['', [Validators.required, Validators.minLength(3)]] });
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] FormBuilder & FormGroup Setup: _____

[2] Custom Validator Implementation: _____

[3] Template Binding & Async Pipe: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **TypeScript Typings:** All models and component inputs strictly typed; no implicit 'any'.
- [ ] **Validation Robustness:** Form validation blocks invalid submissions and displays user feedback.
- [ ] **Template Performance:** *ngFor uses trackBy functions to prevent unnecessary DOM re-rendering.
- [ ] **Reactive Paradigm:** Reactive forms and Observables properly unsubscribed or piped with async.
- [ ] **Component Modularity:** Components maintain clean separation of concerns.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why are Angular Reactive Forms preferred over Template-Driven Forms for complex enterprise applications?*
- *How does using the `async` pipe prevent memory leaks in Angular component subscriptions?*

Reflection Notes:

_____

_____

COMPLETION STATUS:

[ ] COMPLETED (Meets All Solution Criteria)

[ ] NEEDS REVISION (Resubmit Corrected Work)

Evaluator Signature: _____

EVALUATOR & MENTOR REMARKS:

Feedback Notes:

_____

Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Best Practices